TransitOps TRD

Technical Requirements Document (TRD)

TransitOps — Fleet & Logistics Operations Platform

Version: 1.0 **Companion to:** TransitOps PRD v1.0 **Status:** Based on current implementation (backend + frontend codebase review)

---

1. Technology Stack

1.1 Backend

Layer	Technology
Runtime	Node.js 18 (Alpine, per Dockerfile)
Framework	Express 4.19
Database	MongoDB 6.0, via Mongoose 8.4 (ODM)
Auth	`jsonwebtoken` 9.x (JWT), `bcryptjs` 2.4 (password hashing)
Validation	`zod` 4.x (request schema validation)
Caching	`node-cache` (in-process TTL cache, 300s default)
Security middleware	`helmet`, `express-mongo-sanitize`, `xss-clean`, `express-rate-limit`, `cors`
Logging	`winston` (file + console transports) + `morgan` (dev HTTP logging)
Field encryption	`mongoose-field-encryption` (vendored in `utils/encryption.js`)
Docs	`swagger-jsdoc` + `swagger-ui-express`, served at `/api-docs`
Monitoring	`express-status-monitor`, served at `/status`
Email	`nodemailer` (password reset flow)
Process manager	PM2 (`ecosystem.config.js`, cluster mode `-i max` in Docker)

1.2 Frontend

Vanilla HTML5 / CSS3 / JavaScript (ES modules, no build tool). Structure: - `frontend/*.html` — one page per feature (dashboard, trips, vehicles, drivers, fuel-expense, maintenance, reports, analytics, tracking, index/login, error). - `frontend/js/services/api.js` — central fetch wrapper for the REST API. - `frontend/js/services/auth.js` — session/token handling. - `frontend/js/services/offline.js` — offline request queuing. - `frontend/js/pages/*.js` — one controller module per page. - `frontend/js/store.js` — lightweight client-side state. - `frontend/js/utils/` — constants, debounce, helpers, UI components. - `frontend/css/style.css`, `components.css` — glassmorphism design system via CSS variables.

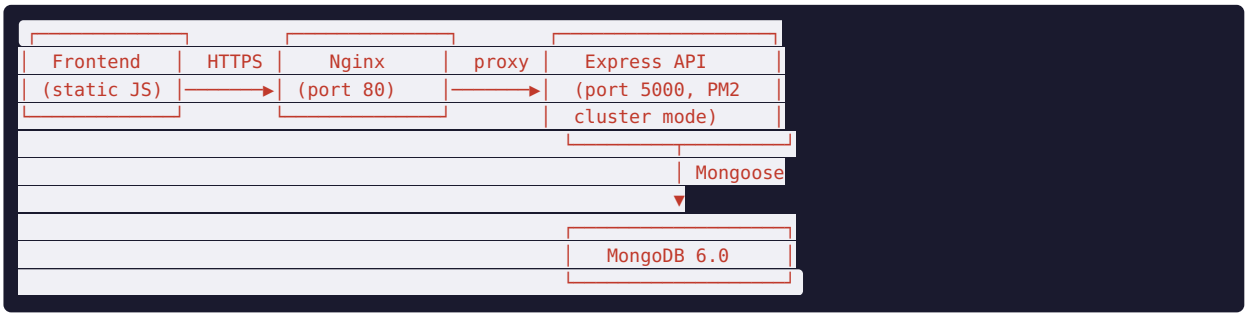
1.3 Infrastructure / DevOps

- **Containerization:** Multi-stage `Dockerfile` (builder installs prod deps → runtime image runs via `pm2-runtime` in cluster mode, exposing port 5000).
- **Local orchestration:** `docker-compose.yml` — 3 services: `api` (Node backend), `mongo` (MongoDB 6.0 with persisted volume), `nginx` (reverse proxy + static frontend host on port 80).
- **Kubernetes:** `k8s/deployment.yaml`, `k8s/service.yaml` for cluster deployment.

- **Reverse proxy:** `nginx/nginx.conf` serves the static frontend and proxies API calls.
- **CI/CD:** GitHub Actions (`.github/workflows/ci.yml`) — matrix build on Node 18.x/20.x, `npm ci` , lint (if present), unit tests (if present) against a test Mongo URI; on `main` pushes, builds (not yet pushes) a Docker image via Buildx.

---

2. System Architecture



2.1 Backend layering

`routes/` → `middleware/` (auth, role, validation, cache) → `controllers/` → `services/` (business logic; e.g. `tripService.js`) → `repositories/` (data access; e.g. `tripRepository.js` , `baseRepository.js`) → `models/` (Mongoose schemas).

Not every module currently uses the full repository/service split — Trip is the reference implementation (`tripService` + `tripRepository`); Vehicle, Driver, Maintenance, Fuel/Expense controllers currently call Mongoose models more directly. This inconsistency is a known refactor target (see §9).

2.2 Request lifecycle (typical protected write)

1. `helmet` , `compression` , `cors` , `express.json` (10kb limit), `mongo-sanitize` , `xss-clean` , rate limiter (100 req / 10 min / IP on `/api/*`).
2. `protect` middleware — verifies JWT, attaches `req.user` (password hash excluded).
3. `authorize(...roles)` — checks `req.user.role` against an allow-list.
4. Zod-based `validate(schema)` or entity-specific validator (`validateDriver` , `validateVehicle`) — 400 on schema violation.
5. Controller → Service (business rules) → Repository → Mongoose model.
6. On success: `clearCache(prefix)` middleware invalidates relevant cached GET responses.
7. `errorMiddleware` (last in chain) formats uncaught errors into a consistent JSON envelope.

---

3. Data Layer

3.1 Database

MongoDB, connection via `config/db.js` using `MONGO_URI` . Default local URI: `mongodb://127.0.0.1:27017/transitops` (dev) or `mongodb://mongo:27017/transitops` (Docker Compose).

3.2 Collections & Key Indexes

Collection	Notable indexes / constraints
`users`	`email` unique + indexed
`vehicles`	`registrationNumber` unique; index on `status` , `vehicleType`
`drivers`	`licenseNumber` unique (stored encrypted)
`trips`	`tripCode` unique (auto-generated `TRIP-XXXXXXXX`); index on `status` , `scheduledDate` (desc)
`fuellogs` , `expenses` , `maintenances`	reference `vehicle` (and optionally `trip`) via ObjectId
`notifications`	optional `userId` (supports broadcast), `referenceId` for

	deep-linking
`auditlogs`	`modelName` + `documentId` + `userId`, `action` enum (CREATE/UPDATE/DELETE/RESTORE)

3.3 Field-Level Encryption

Driver PII (`licenseNumber`, `contactNumber`) is encrypted at rest using AES-256-CBC with a random 16-byte IV per field (vendored implementation of `mongoose-field-encryption` in `utils/encryption.js`). Encryption key comes from `ENCRYPTION_SECRET` env var (SHA-256-derived), falling back to a hardcoded dev default if unset — **must be overridden in every non-dev environment** (see §8, Security Hardening).

3.4 Schema Validation

All entity schemas use Mongoose-level `required`, `enum`, `min`, `unique`, and `trim` validators as the last line of defense; request-level validation (Zod / custom validators) runs first in the middleware chain.

---

4. API Design

4.1 Conventions

- Base path: `/api/v1`
- All responses: `{ success: boolean, message?: string, data?: ..., count?, total?, page?, pages? }`
- Auth: `Authorization: Bearer <JWT>` header
- Pagination: `page`, `limit` (default `limit=100`), `sort` (default `-createdAt`) query params on list endpoints
- Errors: non-2xx responses always include `success: false` and a human-readable `message`

4.2 Endpoint Reference

Auth (`/auth`)

Method	Path	Access	Notes
POST	`/register`	Public	Zod-validated
POST	`/login`	Public	Zod-validated
POST	`/refresh`	Public (valid refresh token)	Issues new access token
POST	`/forgot-password`	Public	Triggers email via nodemailer
POST	`/reset-password`	Public (valid reset token)	
GET	`/me`	Authenticated	Returns current user profile

Vehicles (`/vehicles`)

Method	Path	Access	Cache
GET	`/`	Any authenticated role	300s, key = full URL
POST	`/`	`fleet_manager`	invalidates `/vehicles*`
GET	`/:id`	Any authenticated role	300s
PUT	`/:id`	`fleet_manager`	invalidates `/vehicles*`
DELETE	`/:id`	`fleet_manager`	invalidates `/vehicles*`

Drivers (`/drivers`)

Method	Path	Access	Cache
GET	`/`, `/:id`	Any authenticated role	300s
POST	`/`, PUT `/:id`, DELETE	`safety_officer`	invalidates `/drivers*` and

	`/:id`		`/reports*`

Trips (`/trips`)

Method	Path	Access	Effect
GET	`/` , GET `/:id`	Any authenticated role (implied by `protect`)	Filterable/paginated list
POST	`/`	dispatcher-class role	Runs BR-1/BR-2/BR-4/BR-5 + conflict check; creates `draft` trip
PUT	`/:id/dispatch`	`dispatcherOrManager`	`draft` → `dispatched` ; locks vehicle+driver
PUT	`/:id/complete`	`dispatcherOrManager`	active → `completed` ; frees assets, updates odometer, may emit maintenance notification
PUT	`/:id/cancel`	`dispatcherOrManager`	`draft` / `dispatched` → `cancelled` ; frees assets if reserved
PUT	`/:id/status`	`dispatcherOrManager`	Generic transition endpoint; blocks changes to terminal trips

Maintenance (`/maintenance`)

Method	Path	Access
GET	`/` , GET `/:id`	Any authenticated role
POST	`/`	`managerOrSafety` (`fleet_manager` , `safety_officer`) — enforces BR-9
PUT	`/:id`	`managerOrSafety`
PUT	`/:id/close`	`managerOrSafety` — rejects if already closed

Fuel & Expenses (`/fuel-expenses`)

Method	Path	Access
GET	`/live-rates`	Any authenticated role
GET	`/fuel` , GET `/expenses`	Any authenticated role
POST	`/fuel` , POST `/expenses`	`analystOrManager` (`financial_analyst` , `fleet_manager`)

Reports (`/reports`)

Method	Path	Access	Cache
GET	`/dashboard`	Any authenticated role	300s
GET	`/fuel-efficiency`	Any authenticated role	300s
GET	`/operational-cost`	Any authenticated role	300s
GET	`/driver-performance`	`financial_analyst` , `fleet_manager`	300s
GET	`/export/trips`	`financial_analyst` , `fleet_manager`	CSV stream download

Notifications (`/notifications`)

Method	Path	Access
GET	`/`	Authenticated
PUT	`/:id/read`	Authenticated

AI (`/ai`) — placeholder

Method	Path	Status
POST	`/optimize-route`	Stub handler, not integrated with a real optimizer
POST	`/assistant`	Stub handler, not integrated with an LLM backend

Misc

- `GET /api/v1/health` — liveness check, returns status + timestamp.
- `GET /api-docs` — Swagger UI.
- `GET /status` — `express-status-monitor` dashboard.

---

5. Business Logic Implementation Detail

5.1 Trip Service (`services/tripService.js`)

Central place for trip business rules. Key methods: - `getTrips(query)` — builds a Mongo filter from status/priority/vehicle/driver/date-range/search, paginates via `tripRepository` . - `_checkSchedulingConflict(vehicleId, driverId, scheduledDate, eta, excludeTripId?)` — queries for any non-terminal trip whose window `[scheduledDate, eta]` overlaps the requested window for the same vehicle **or** driver; throws with the conflicting `tripCode` . - `createTrip(data)` — validates vehicle existence/not-retired (BR-1), driver existence/not-suspended/license-not-expired (BR-2/BR-4), cargo ≤ capacity (BR-5), then scheduling conflict, then persists. - `dispatchTrip(id)` — re-validates status `draft` , vehicle `available` (BR-3), driver `available` , license still valid, capacity still valid; sets vehicle/driver to `on_trip` ; trip → `dispatched` . - `completeTrip(id, {actualDistance, fuelConsumed, revenue})` — only from `dispatched` / `in_transit` / `delivered` ; frees vehicle (sets `available` , increments `odometer`), frees driver; if `odometer >= nextMaintenanceMileage - 500` , creates a `warning` Notification; trip → `completed` , `completedAt` stamped. - `cancelTrip(id)` — only from `draft` / `dispatched` ; frees assets if they had been reserved. - `updateTrip(id, data)` — only while `draft` ; re-runs scheduling-conflict check excluding the current trip. - `updateTripStatus(id, status)` — generic transition; rejects further changes once `completed` / `cancelled` ; auto-frees or auto-reserves vehicle/driver depending on the direction of the transition.

5.2 Maintenance Controller (`controllers/maintenanceController.js`)

- `createMaintenance` — BR-9: rejects if vehicle status is `on_trip` or `retired` ; otherwise creates the record (vehicle should transition toward `in_shop` per business rule).
- `closeMaintenance` — rejects if record is already `status: closed` .

5.3 Auto-generated Trip Code

`Trip` schema `pre('save')` hook generates `TRIP-<8 uppercase hex chars>` from `crypto.randomUUID()` when a new document has no `tripCode` .

5.4 Audit Logging

`utils/auditLogger.js` 's `auditLog(action, modelName, documentId, userId, changes)` is invoked from trip controller actions (create/dispatch/complete/cancel/status-update) to write an `AuditLog` document capturing who did what.

---

6. Caching Strategy

- Implementation: `node-cache`, in-process (single-instance) TTL cache, default 300s.
- Cache key: full request URL (`req.originalUrl`) — includes query string, so filtered list requests cache independently.
- Only `GET` responses with 2xx status are cached; the response body is deep-cloned via `JSON.parse(JSON.stringify(body))` before storage to avoid caching live Mongoose document references.
- Invalidation: `clearCache(prefix)` middleware runs after successful writes and deletes all cache keys starting with the given prefix (e.g. `/api/v1/vehicles`).
- **Known limitation:** because the cache is in-process, it does **not** stay consistent across multiple PM2 cluster workers or multiple pods — a write on worker A does not invalidate cache entries held by worker B. Acceptable for the hackathon/single-node deployment; would need Redis (or disabling per-worker cache) for a true multi-instance production deployment.

---

7. Error Handling & Logging

- `middleware/errorMiddleware.js` is the terminal Express error handler — normalizes thrown errors (including Mongoose `ValidationError` / `CastError` /duplicate-key `11000`) into `{ success: false, message }` with an appropriate status code (`err.statusCode` where set, else 500).
- Controllers largely follow a pattern of `try { ... } catch (err) { err.statusCode = 400/404; next(err) or res.status(...).json(...) }`.
- `utils/logger.js` (Winston) writes structured logs to `logs/combined.log` (all levels) and `logs/error.log` (errors), plus console in development via `morgan('dev')`.
- Service-layer methods (e.g. `tripService`) call `logger.info(...)` on key lifecycle events (`create_trip`, `dispatch_trip`, `complete_trip`, `cancel_trip`, `update_trip_status`) with structured metadata.

---

8. Security

Concern	Implementation
Transport	Helmet sets HSTS (`max-age=31536000; includeSubDomains; preload`) and `frameguard: deny`; TLS termination expected at Nginx/ingress, not in-app
AuthN	JWT (`JWT_SECRET`, `JWT_EXPIRE=7d` by default); refresh-token endpoint
AuthZ	Per-route `authorize(...roles)` allow-lists (RBAC)
Passwords	bcrypt, salt rounds = 12
PII at rest	AES-256-CBC field encryption for driver license number & contact number
Injection	`express-mongo-sanitize` (NoSQL injection) + `xss-clean` (script injection) on all bodies/params/query
Payload size	`express.json({ limit: '10kb' })`
Rate limiting	100 requests / 10 minutes / IP on all `/api/*` routes
CORS	Explicit allow-list; production restricted to `FRONTEND_URL` env var, dev allows localhost:5500/3000
Startup safety	Server refuses to boot if `JWT_SECRET` is unset (`process.exit(1)`)

8.1 Hardening Recommendations for Production

- Set `ENCRYPTION_SECRET` explicitly — do not rely on the hardcoded fallback in `models/Driver.js`.
- Move the field-encryption implementation to the maintained `mongoose-field-encryption` npm package rather than the vendored copy in `utils/encryption.js`, and confirm it is kept patched.
- Set `FRONTEND_URL` and restrict CORS methods/headers per environment.
- Consider moving the in-process cache to Redis if scaling beyond a single Node process (PM2 cluster / multiple pods), to keep cache invalidation consistent.
- Add automated tests to satisfy the CI pipeline's `npm test` / `npm run lint` steps (currently `--if-present`, i.e. effectively skipped if absent).

9. Known Technical Debt / Gaps

1. **Inconsistent architecture layering** — Trip uses service+repository; Vehicle/Driver/Maintenance/Fuel-Expense controllers talk to Mongoose models more directly. Recommend extending the `baseRepository` / `baseService` pattern uniformly.
2. **Role constant drift** — `constants/index.js` defines `ROLES: ['admin', 'dispatcher', 'fleet_manager', 'driver', 'analyst']`, which doesn't match the roles actually enforced across routes (`fleet_manager`, `dispatcher`, `safety_officer`, `financial_analyst`, `viewer`). Should be reconciled into a single source of truth.
3. **AI routes are stubs** — `/ai/optimize-route` and `/ai/assistant` exist as route placeholders with no real backing logic.
4. **Single-instance cache** — `node-cache` cache/invalidation does not propagate across PM2 cluster workers or Kubernetes replicas.
5. **CI test/lint steps are conditional** (`--if-present`) — no test suite currently ships in the repo, so CI currently only validates that `npm ci` succeeds.
6. **Docker image doesn't include the frontend** — the frontend is served separately via the `nginx` service in Compose / would need its own static hosting in the Kubernetes manifests.

10. Deployment Topology

10.1 Docker Compose (reference/dev-prod-like)

- `api` — built from root `Dockerfile`, port 5000, depends on `mongo`.
- `mongo` — official `mongo:6.0` image, persisted via named volume `mongo_data`.
- `nginx` — serves `frontend/` statically and reverse-proxies to `api`, port 80.
- All services share the `transitops_network` bridge network.

10.2 Kubernetes

- `k8s/deployment.yaml` — Deployment spec for the API container.
- `k8s/service.yaml` — Service exposing the Deployment internally/externally.
- *(No HPA, ConfigMap/Secret manifests, or Ingress resource currently checked in — would be required for a production-grade cluster rollout.)*

10.3 Process Management

- Inside the container, PM2 runs the API in cluster mode (`pm2-runtime backend/server.js -i max`), auto-scaling worker processes to available CPU cores and restarting on crash.
- `ecosystem.config.js` at the repo root defines the PM2 app config for non-Docker deployment (e.g. bare-metal/VM).

10.4 Required Environment Variables

Variable	Purpose	Required
`PORT`	API port (default 5000)	No
`MONGO_URI`	MongoDB connection string	Yes
`JWT_SECRET`	JWT signing secret	**Yes — hard fail on boot if missing**
`JWT_EXPIRE`	Access token TTL (e.g. `7d`)	No (has default)
`NODE_ENV`	`development` / `production`	No
`FRONTEND_URL`	Allowed CORS origin in production	Recommended in prod
`ENCRYPTION_SECRET`	Key for driver PII field encryption	**Strongly recommended** (insecure default otherwise)

Separate env files are maintained per environment: `.env.development`, `.env.staging`, `.env.production` (frontend-level), and `backend/.env` / `backend/.env.example` (backend-level).

11. Testing & Quality Gates (current state)

- CI runs on Node 18.x and 20.x matrix for every push/PR to `main` / `develop`.
- Steps: install (`npm ci`) → lint (if a lint script exists) → unit tests (if a test script exists) → (on `main`) Docker image build via Buildx (not yet pushed to a registry).
- **Gap:** no test files currently exist in the repository; adding a unit/integration test suite (e.g. Jest + Supertest against an in-memory Mongo instance) is recommended before production hardening.

This TRD should be read alongside the PRD for business context. Say the next document name (e.g. “api docs”, “er diagram”, “user manual”) to continue.